

Algoritmos y Estructuras de Datos

1er semestre - 2026

Árbol Binario de Búsqueda

- Se dice que un árbol binario es de búsqueda si está organizado de forma que:
 - para cada nodo ti todas las claves del subárbol izquierdo de ti son menores que la clave de ti y todas las claves del árbol derecho son mayores.
- Si el árbol tiene n nodos y está balanceado, su altura será de $\log(n)$
- Una búsqueda en este caso puede tomar $\log(n)$ comparaciones o menos.

Árbol Binario de Búsqueda

```
public interface TDAArbolBinario<T>{  
    boolean isVacio();  
    boolean insertar(Comparable<T> dato);  
    T buscar(Comparable<T> criterio);  
    T eliminar(Comparable<T> criterio);  
    void preOrder(Consumer<T> consumer);  
    void inOrder(Consumer<T> consumer);  
    void postOrder(Consumer<T> consumer);  
    // Hay más operaciones  
}
```

Árbol Binario de Búsqueda

```
public class ABBImpl<T> implements TDAArbolBinario<T>{

    private TDAElemento<T> raiz;

    // Hay más operaciones
    @Override
    public T buscar(Comparable<T> criterioBusqueda){
        if (raiz != null){
            TDAElemento<T> resultado = raiz.buscar(criterioBusqueda);
            if (resultado == null) return null;
            return resultado.getDato();
        }
        return null;
    }
}
```

Árbol Binario de Búsqueda

```
public interface TDAElemento<T>{
    void insertar(Comparable<T> dato);
    TDAElemento<T> buscar(Comparable<T> criterioBusqueda);
    TDAElemento<T> eliminar(Comparable<T> criterio);
    boolean esHoja();
    boolean isVacio();
    void inOrden(Consumer<T> consumer);
    void preOrden(Consumer<T> consumer);
    void posOrden(Consumer<T> consumer);
    TDAElemento<T> getHijoIzq();
    TDAElemento<T> getHijoDer();
    void setHijoIzq(TDAElemento<T> nodo);
    void setHijoDer(TDAElemento<T> nodo);
    // hay más operaciones
}
```

Árbol Binario de Búsqueda

```
public class ElementoABBImp1<T> implements TDAElemento<T>{  
    private T dato;  
    private TDAElemento<T> hijoIzq;  
    private TDAElemento<T> hijoDer;  
  
    // Están las operaciones de la interfaz  
    // y alguna otra más  
}
```

TArbolBB – Buscar Etiqueta

```
1: // Estamos en implementación de TDAArbolBinario<T>
2: procedure BUSCAR(Comparable<T>criterio)
3:     if esVacio() then
4:         return null
5:     else
6:         nodo  $\leftarrow$  raiz.buscar(criterio)
7:         if nodo == null then
8:             return null
9:         else
10:            return nodo.getDato
11:        end if
12:    end if
13: end procedure
```

Árbol Binario de Búsqueda

```
1: // Estamos en implementación de TDAElemento<T>
2: procedure BUSCAR(Comparable<T>criterio)
3:   resultado  $\leftarrow$  null
4:   if criterio.compareTo(this.dato) == 0 then
5:     resultado  $\leftarrow$  this
6:   else
7:     if criterio.compareTo(this.dato) < 0 then
8:       // lo que buscamos está a la izquierda
9:       if hijoIzq  $\neq$  null then
10:        resultado  $\leftarrow$  hijoIzq.buscar(criterio)
11:      end if
12:    else
13:      // lo que buscamos está a la derecha
14:      if hijoDer  $\neq$  null then
15:        resultado  $\leftarrow$  hijoDer.buscar(criterio)
16:      end if
17:    end if
18:  end if
19:  return resultado
20: end procedure
```

Inserción en árboles binarios de búsqueda

```
1: // Estamos en implementación de TDAArbolBinario<T>
2: procedure INSERTAR(Comparable<T>dato)
3:     // retorna TDAElemento<T>
4:     if esVacio() then
5:         raiz  $\leftarrow$  nuevo TDAElemento con dato
6:     else
7:         raiz.insertar(dato)
8:     end if
9:     return true
10: end procedure
```

Inserción en árboles binarios de búsqueda

```
1: // Estamos en implementación de TDAElemento<T>
2: procedure INSERTAR(Comparable<T>nuevoDato)
3:   if nuevoDato.compareTo(this.dato) > 0 then
4:     // nuevoDato es mayor al nodo actual
5:     if hijoDer == null then
6:       hijoDer ← crear ElementoABBImp con nuevoDato
7:     else
8:       hijoDer.insertar(nuevoDato)
9:     end if
10:  else if nuevoDato.compareTo(this.dato) < 0 then
11:    // nuevoDato es menor al nodo actual
12:    if hijoIzq == null then
13:      hijoIzq ← crear ElementoABBImp con nuevoDato
14:    else
15:      hijoIzq.insertar(nuevoDato)
16:    end if
17:  end if
18: end procedure
```

Implementación de recorridos en ABB

- Preorden
- Inorden
- Postorden

TArbolBB - Inorden

```
// estamos bajo la implementación de TDAArbolBinario<T>
public String imprimirInOrden(){
    StringBuilder sb = new StringBuilder();
    if (this.raiz == null){
        sb.append("Árbol vacío");
    }else{
        this.raiz.inOrden(nodo -> {
            sb.append(nodo.getDato());
        });
    }

    return sb.toString();
}
```

TElementoAB - Inorden

```
// estamos bajo la implementación de TDAElemento<T>
public void inOrden(Consumer<TDAElemento<T>> consumer){

    if (this.hijoIzq != null) this.hijoIzq.inOrden(consumer);

    consumer.accept(this);

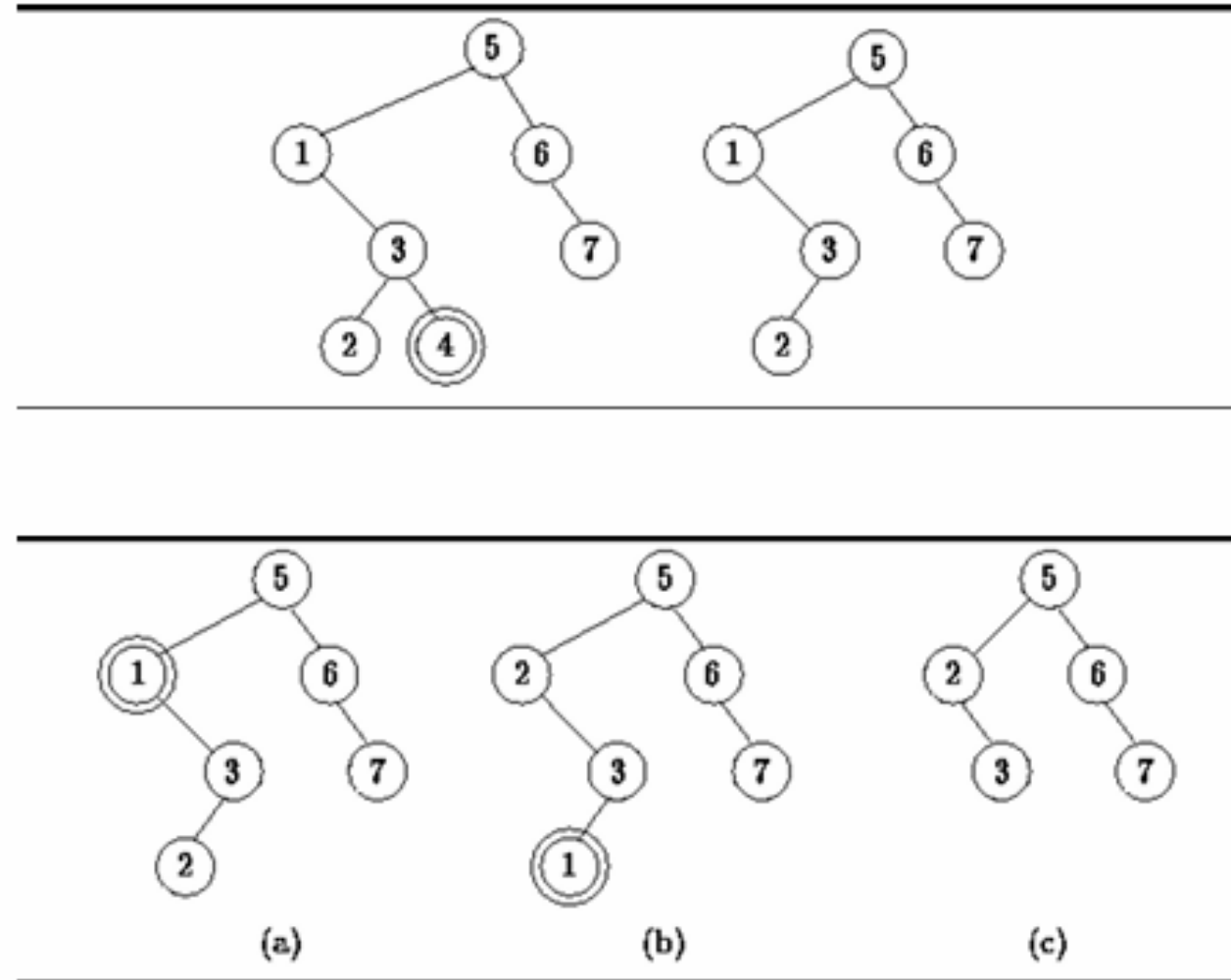
    if (this.hijoDer != null) this.hijoDer.inOrden(consumer);
}
```

Búsqueda y eliminación en árboles binarios de búsqueda

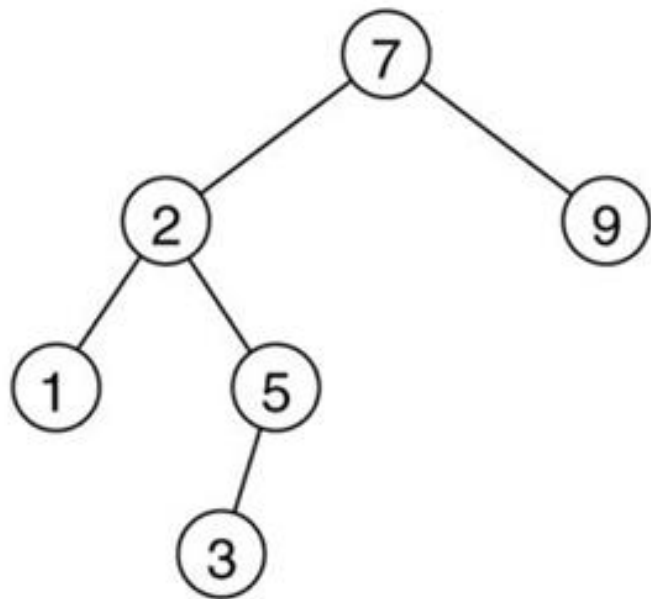
Búsqueda y eliminación en árboles binario de búsqueda

- Se busca el nodo con la etiqueta a eliminar.
- Si no existe un nodo con esa etiqueta, la eliminación es infructuosa.
- Si existe y es una hoja, se elimina el nodo directamente.
- Si no es una hoja, pero le falta alguno de sus dos sub árboles, la operación es sencilla, ya que basta con una reasignación de referencias.
- Si el nodo a eliminar es un nodo interno completo, deben sustituirse sus datos y etiqueta por el los del nodo de mayor etiqueta de su sub árbol izquierdo (su “inmediato anterior” en el orden lexicográfico dado).
- Luego se elimina ese elemento, que será una hoja, o le faltará el sub árbol derecho.

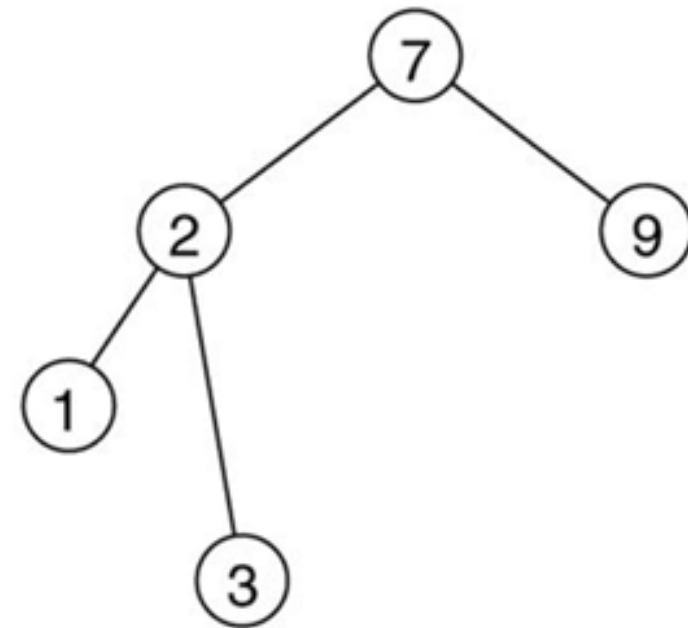
Eliminación



Eliminación del nodo 5 con 1 hijo: (a) antes y (b) después

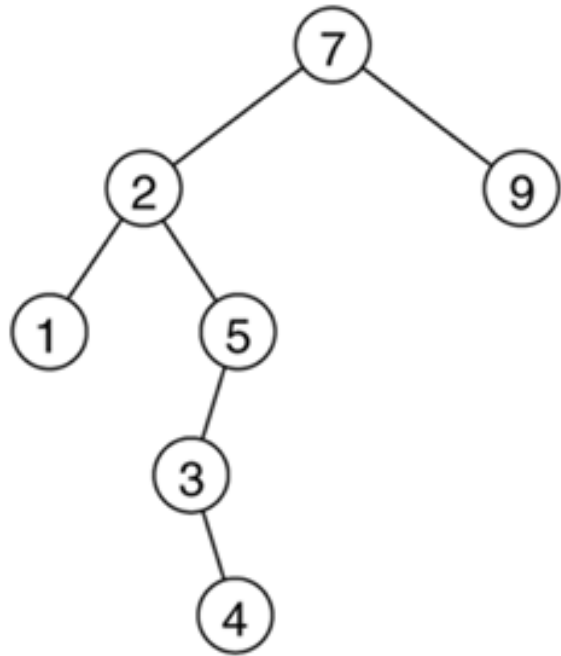


(a)

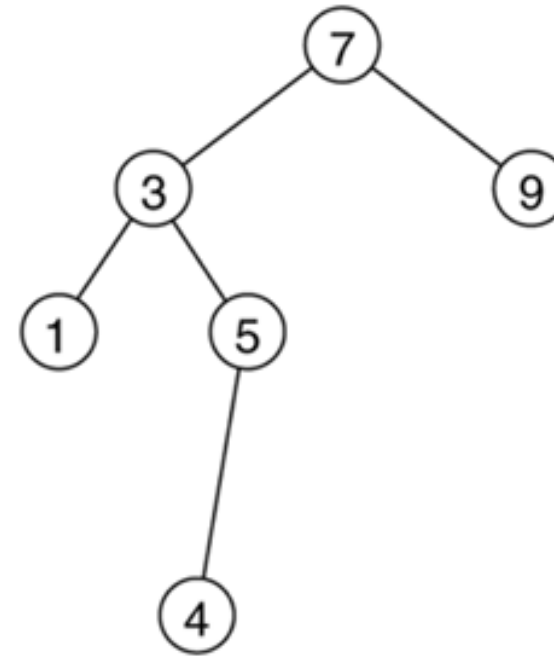


(b)

Eliminación del nodo 2 con dos hijos: (a) antes y (b) después



(a)



(b)

Búsqueda y eliminación en árboles binarios de búsqueda

```
1: // Estamos bajo la implementación de TDAArbolBinario<T>
2: procedure ELIMINAR(Comparable<T>criterio)
3:   if this.raiz  $\neq$  null then
4:     this.raiz = this.raiz.eliminar(criterio)
5:   end if
6: end procedure
```

Eliminación en árboles binarios de búsqueda

```
1: // Estamos bajo la implementación de TDAElemento<T>
2: procedure ELIMINAR(Comparable<T>criterio)
3:     // retorna un TDAElemento<T>
4:     if criterio.compareTo(this.dato) < 0 then
5:         // está del lado izquierdo
6:         if this.hijoIzq ≠ null then
7:             this.hijoIzq ← this.hijoIzq.eliminar(criterio)
8:         end if
9:         return this
10:    else if criterio.compareTo(this.dato) > 0 then
11:        // está del lado derecho
12:        if this.hijoDer ≠ null then
13:            this.hijoDer ← this.hijoDer.eliminar(criterio)
14:        end if
15:        return this
16:    end if
17:    return quitarNodo()
18: end procedure
```

```
1: // Estamos bajo la implementación de TDAElemento<T>
2: procedure QUITARELNODO
3:     // retorna un TDAElemento<T>
4:     if this.hijoIzq == null then
5:         return this.hijoDer
6:     else if this.hijoDer == null then
7:         return this.hijoIzq
8:     else
9:         // es un nodo completo
10:        elHijo  $\leftarrow$  this.hijoIzq
11:        elPadre  $\leftarrow$  this
12:        while elHijo.hijoDer  $\neq$  null do
13:            elPadre  $\leftarrow$  elHijo
14:            elHijo  $\leftarrow$  elHijo.hijoDer
15:        end while
16:        if elPadre  $\neq$  this then
17:            elPadre.hijoDer = elHijo.hijoIzq
18:            elHijo.hijoIzq = hijoIzq
19:        end if
20:        elHijo.hijoDer = hijoDer
21:        return elHijo
22:    end if
23: end procedure
```

Análisis de Búsqueda e Inserción en Árbol Binario de Búsqueda

- Los algoritmos de inserción y búsqueda analizados no controlan el balanceo del árbol.
- Se desconoce a priori la forma en que el árbol ha de crecer.
- Si el árbol estuviera balanceado, para encontrar una clave se precisarían aproximadamente $\log n$ comparaciones.
- El peor caso se da para $n/2$ comparaciones.
- El problema es encontrar la cantidad promedio de comparaciones.
- n claves, $n!$ árboles correspondientes a $n!$ permutaciones de claves.

Ejercicios Árboles Binarios de Búsqueda

- ¿Qué es un árbol binario de búsqueda?
Desarrolle el algoritmo de búsqueda correspondiente y evalúe su rendimiento.
- Desarrolle el algoritmo de eliminación en árbol binario de búsqueda y evalúe su rendimiento
- Muestre el resultado de insertar los elementos 3,1,4,6,9,2,5 y 7 en un árbol binario de búsqueda inicialmente vacío. Muestre después el resultado de eliminar la raíz